

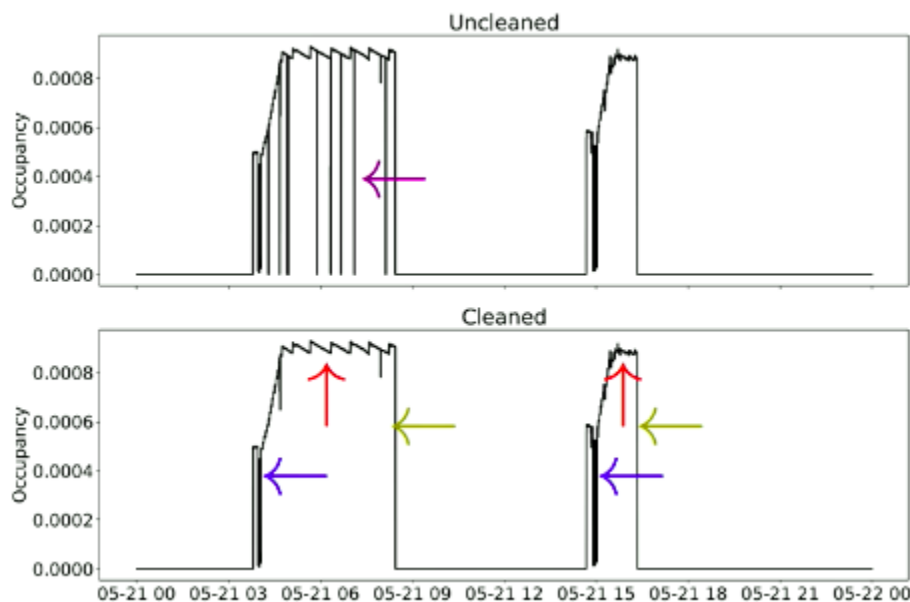
Research and resources notes:

How to LHC works

1. 99% of the Tunnel (Separate Paths):
For almost the entire 27 km circumference, Beam 1 and Beam 2 travel in two completely separate vacuum pipes running side-by-side, about 19 cm apart.
2. The 4 Intersection Points:
As the two pipes approach one of the 4 experiment caverns (ATLAS, CMS, ALICE, or LHCb), special steering magnets bend the two pipes together into a single shared section of vacuum pipe so the two beams cross paths directly in the center of the detector.
3. After the Intersection:
Right after passing through the detector, another set of magnets pulls the two beams back apart into their separate parallel pipes until they reach the next experiment cavern.

1) Anomaly detection at ATLAS pixel detector (presentation by Yang, Kia-Jung - Georg August Universitaet Goettingen (DE))

- Currently Shifters/Experts sitting at ATLAS control room monitoring the condition of detectors by checking anomaly in the data
- Use ML to detect outliers of various time series data coming from the DAQ and DCS, to reduce no of shifters and predict future failure



- First, around 300 trillion protons are added to the ring and the ring is closed and the magnet starts to accelerate the protons into two beams. Once too many of the supplied protons collide and separate from beam, the beam is dumped into massive graphite blocks. The whole cycle is called a LHC Fill.
- Run- a period of continuous, stable *data-taking* by ATLAS within a fill.

- Each of the two beams is made of thousands of bunches (proton cloud), Every 25 nanoseconds, one bunch from Beam 1 and one bunch from Beam 2 cross paths inside ATLAS (bunch crossing). One collision event is the 3D data recorded in that 25 nanosecond.
- An occupancy of 0.0008 (8×10^{-4}) means that in a given collision event, approximately 8 out of every 10,000 pixel channels registered a hit
- Emittance Scan (Purple Arrow): beam separation is done at start to know the occupancy when the two beams path have no overlap (so no collision) so it occupancy fall to near 0
- Luminosity Leveling (Red Arrow): since the supplied protons decreases (due to collision and separation from beam) as time pass, to make the occupancy in same level for the entire experiment, they start out without complete path overlap of two beams and slowly adjust magnetic field to overlap it more and more.
- Beam Dump (Green Arrow): extraction and disposal of proton beams out of the LHC ring into absorber blocks.

Ways to make clean the data

- Occupancy drops to 0 from time to time (called artifacts) because the detector's readout pipeline temporarily stopped recording, lost sync, or discarded the data. These dont have any important meaning so we can remove them. We remove them by using the fact that these dumps are correlated with another variable called A1 trigger rates.
- Throw away all zeros in the occupancy

Ways to analyze the data to predict future failure

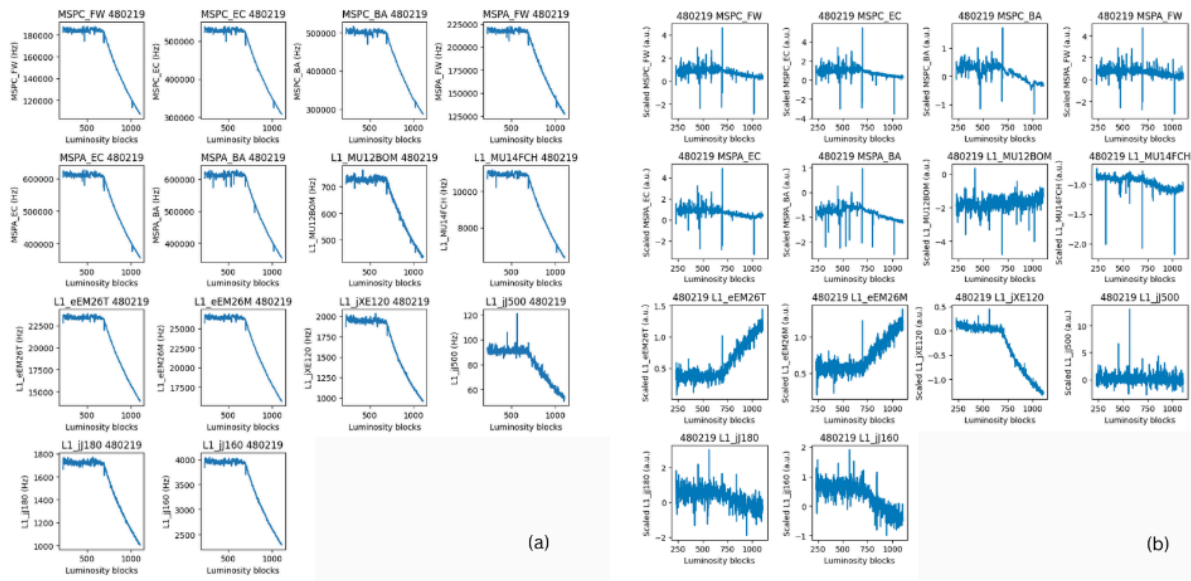
- Find when are trends and fluctuation normal and when not
- Find correlation of trends and fluctuation with other variables

2) Automating ATLAS control room anomaly detection with deep learning (research paper)

- Since physically impossible to write all that data of each collision to disk(because how huge it is), use trigger and data acquisition (TDAQ) system which filters down the data just to what is most relevant. The first pass in this filtering is the Level-1 (L1) trigger. L1 Trigger consists of hundreds of “trigger items” to detect specific particles. L1 Rate for any specific item is simply how many times per second (or any time frame like 4sec or 5sec) that trigger condition (condition that it is the specified particle) is satisfied. For three items, the data looks like below

```
eEM26T: (timestamp1, value), (timestamp2, value), (timestamp3, value)...\njj500:  (timestamp_a, value), (timestamp_b, value)...\npileup: (timestamp_x, value), (timestamp_y, value)...
```

- The model used is a predictive long short-term memory autoencoder that takes in time-series data of multiple L1 rates and via unsupervised learning, learns to predict how those rates will change such that the level of error between the prediction and real data can be used to classify the data as clean or anomalous.
- The model can easily be adapted to run in real-time.
- Data quality monitoring (DQM) is generally divided into two areas:
 - Online DQM involves monitoring the data in real time and allows for intervention to correct problems that may arise.
 - Offline DQM is a more rigorous sweep done after acquisition to review the quality of data that has been stored away and its usefulness for analyses
- Many current approaches of DQM on anomaly detection rely on autoencoders, especially variational autoencoders
- Luminosity: Measures how many protons are squeezed into the interaction region per second.(Interaction region is 3D volume inside pipe where proton beams smash each other). It is the geometric center of the ATLAS detector.
- Pile-up : The average number of individual proton-proton collisions that happen in a single bunch crossing.
- Luminosity Block: unit of time used by LHC experiments (around 60s). In this time window luminosity, beam alignment, pile-up are considered to be constant.

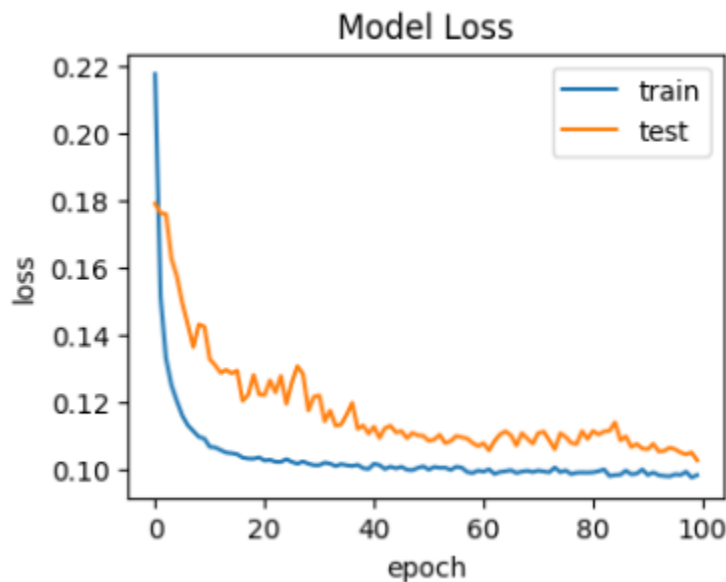


- To make the left side graph, look at which LB window each timestamp from L1 trigger fall into and convert the raw input from L1 trigger into something like

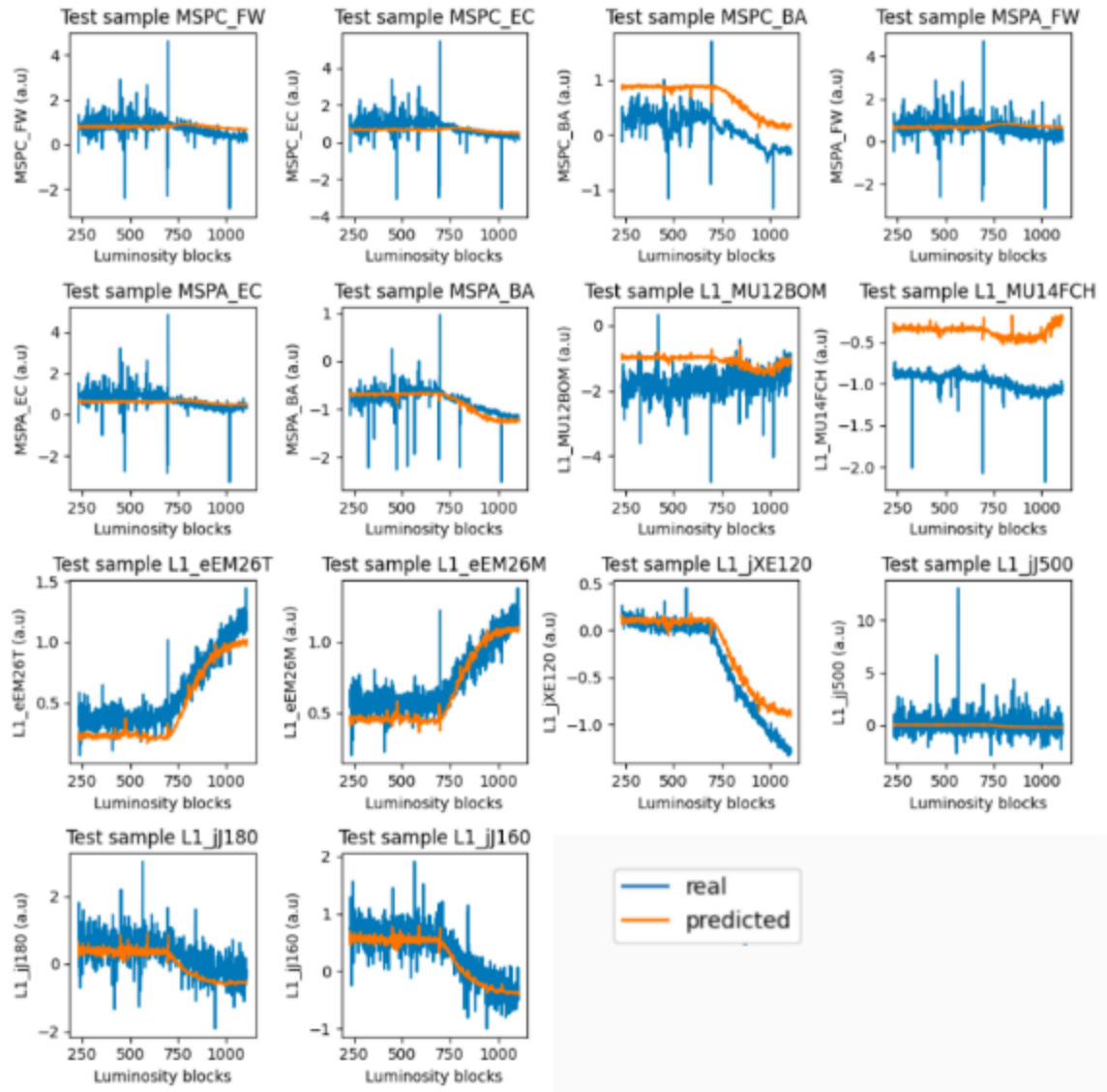
LB	eEM26T	jj500	MSPC_FW	pileup
1	18500	1050	145000	58.2
2	18700	1080	143000	57.9
3	19100	1020	146500	58.0

- some LBs have multiple readings for one feature (so you average), and some LBs end up with zero readings for a feature (filled by interpolation)
- The pileup drops in the second half as more and more protons collide and get removed from the beam and there can be no more overlapping of beam to make it constant.
- We transform the left figures into right by dividing each feature by pileup (because the stable at first and going down is predictable, uninteresting pattern that has nothing to do with detector health) and scaling by subtracting median and dividing it by Interquartile range
- Their model uses 14 features only (**could be improved**)
- Features like "busy rates"(How frequently a hardware memory buffer or data queue reaches 100% full capacity.) and "empty rates"(How often bunch crossings occur with zero collisions) were not used as input because they were hard to predict and hence triggered alarms a lot
- They explored the use of autoencoders and long short-term memory (LSTM) networks using TensorFlow
- Since autoencoder take one row at a time as input, can only find point anomalies using MSE between input and output

- Then they used Non-predictive LSTM Autoencoder where the encoder/decoder layers were LSTM layers instead and using reconstruction MSE. Causes higher false positive rate.
- Final model used here was Predictive LSTM autoencoder where the input is 5 LBs (5 rows) and it tries to predict 1 LB in the future for each LB (so again 5 rows shifted down by one)
- Here model training was done using data from 10 runs (476875, 476428, 476276, 476785, 476718, 479279, 479269, 479239, 480188, 480197)(around 8000 data point) **(could be improved)**
- Here model testing was done using data from 1 run (480219) (around 900 data points)

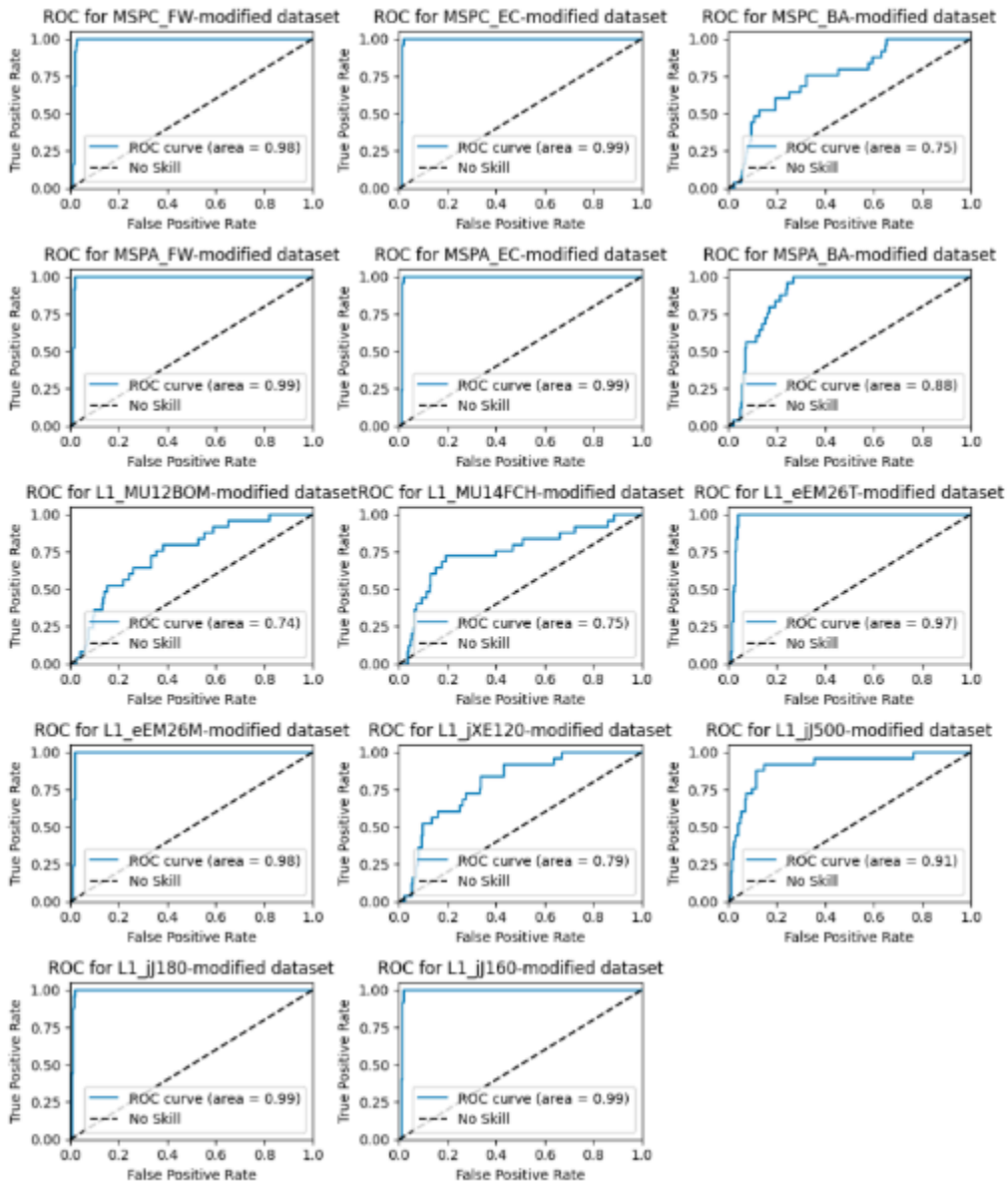


- MSE squares the error between predicted and actual values so massively amplifies large errors. Few outliers in the training set contribute to large loss. Huber loss behaves like MSE (quadratic) for small errors, linear for large so use Huber loss to get small difference between test and train loss at the end. Could have tried to remove outliers in training data but not seem necessary after robust scaling and using huber loss.
- Fig shows how prediction aligns with test data of a certain feature



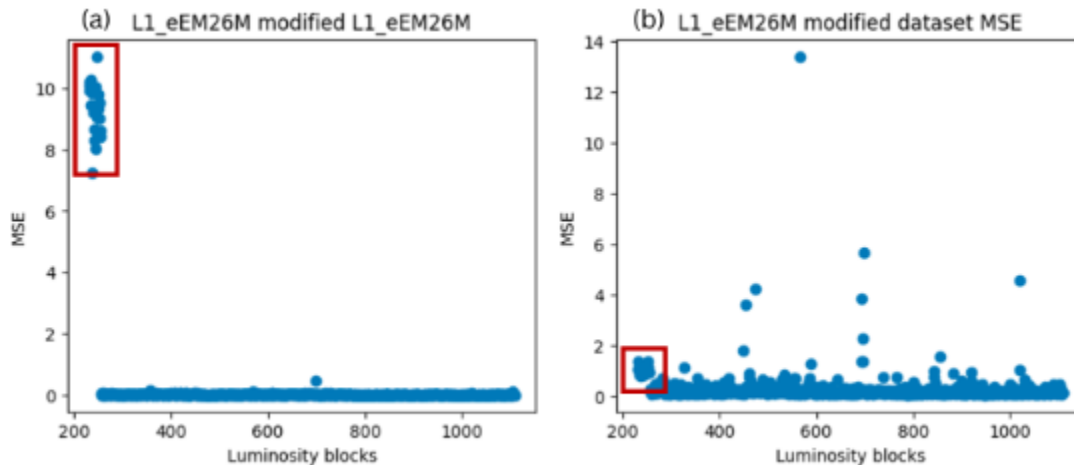
- notably in the cases of MSPC_BA and L1_MU14FCH, the predicted results are shifted in the y-direction from the real data (because conditions changed in the test run). A way to resolve this could be to retrain the model at regular intervals during test so that after say 30 LB of the new run, the model would retrain and by incorporating the data of the present run into training, it would adapt to any changes. We would still expect to see the shift between real and predicted at the beginning of test run, but with each retraining, we would hope to see the two curves match better and better. **(could be improved)**
- At tail end of L1_jXE120 it seems like predicted and actual is separate, could be due to us trying to squash the impact of outliers, we inadvertently squash the impact of the real low data since it is at an extreme. To address this, we recommend implementing artwise scaling procedures such that the portion of the run where pileup is flat is scaled separately from the region where pileup is decaying. **(could be improved)**

- They modified the test data to introduce artificial anomalies. They generated 14 different modified datasets each corresponding to a change in one of the 14 features by increasing the feature by 5% over the first 30 LBs (**could be improved**).
- For each modified dataset, they plot a Receiver Operating Characteristic (ROC) curve, which shows the tradeoff between two rates as you vary the model's anomaly-classification threshold from very strict to very lenient. diagonal dashed line represents random guessing. The closer a curve hugs the top-left corner, the better.

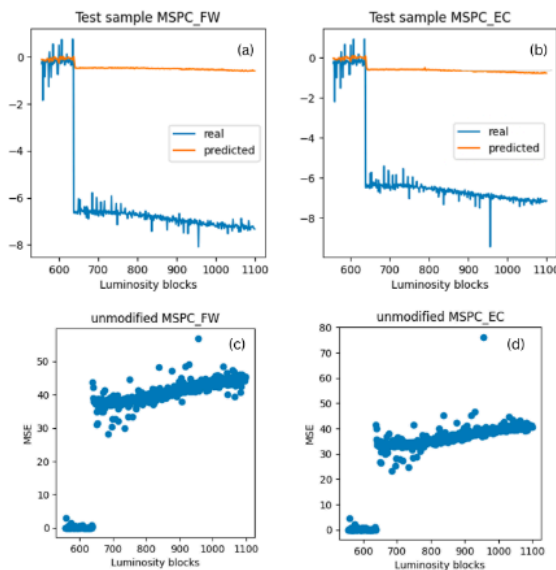


- also explored how these ROC curve results change with different sizes of anomalies, affecting different numbers of LBs, and occurring at different points in the run (beginning, middle, or end). Performance is slightly worse in the middle of the run and rather improved at the end of the run.

- The distinction between anomalous and clean data points is much better defined in Figure (a) where the MSE for the changed feature is plotted. However, the actual classification in our current model is based on the average MSE across all features as seen in Figure (b). Fig (b) is not that good cuz the test data we modified may have had some anomalies of its own even before we implemented our shift



- They have done some initial exploration of deciding if anomalous or clean by feature plots (as seen in Figure a) with custom thresholds set for each feature such that if a data point exceeds the threshold for any feature it is labeled as anomalous. (this improved our true positive detection from 24/25 to 25/25 but at a cost of an increased false positive rate from 19/859 to 48/859) (could be picking up actual anomalies of test data)
- To see if the model can handle real types of anomaly due to detector failure they used a case study run in which a sector of the muon end cap (1/32 of the full muon trigger detector) was disabled.shutoff occurring around LB number 650.



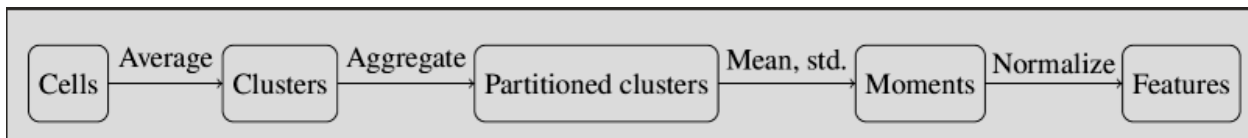
- So the model seems to find real types of anomaly too but this is not how we want model to work in real time in control room because the model would keep on warning for the

rest of the run. We just want to be warned of the anomaly when the initial change occurs, but then we would like the model to adapt to the new conditions so we can continue to use it to look for other anomalies that may pop up. Solution could be to retrain at regular interval throughout the course of the run, allowing it to take in the new data and make predictions that better reflect the current conditions **(could be improved)**

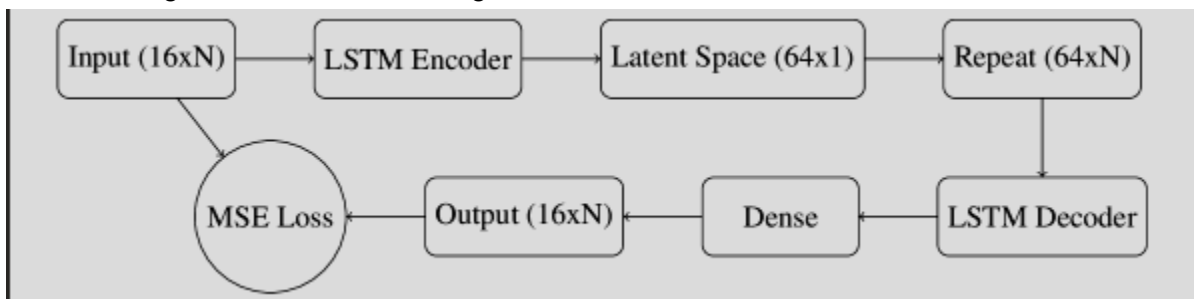
- Future research **(Could be improved)**
 - a. performance of our model should be bench-marked against non-ML models for example using a simple algorithm of the absolute difference between consecutive data points which we would expect to be good at finding point fluctuations
 - b. create a test set with a wider range of anomaly classes to assess performance.
 - c. Implementing piece-wise normalization such that the region corresponding to constant pileup is scaled separately from the region with decaying pileup or separate the constant and decaying pileup regions altogether and train two models separately
 - d. scaling up to a more extensive feature set
 - e. retrains at regular periods
 - f. incorporating reinforcement learning could be beneficial. For a certain range of MSE where the model is not especially confident if the data indicates an anomaly or not, it could flag a shifter and they could provide a label for the data which would allow the model to improve its future decisions
 - g. room for exploration in terms of the actual architecture of the model (transformers could be used instead of LSTM) (to reduce variation in performance depending on the feature the anomaly occurred in)

3) Autoencoder-based time series anomaly detection for ATLAS Liquid Argon calorimeter data quality monitoring (research paper)

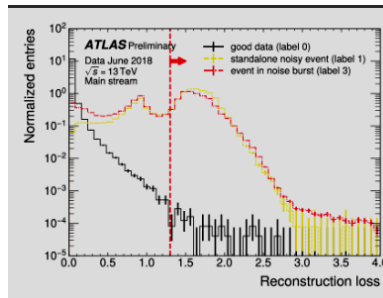
- long short-term memory autoencoder-based algorithm for detecting anomalies in ATLAS Liquid Argon calorimeter data
- Trained on good-quality data
- Validation is performed using a known short-term issue of noise bursts
- existing algorithms focus on well-known detector issues, whereas an AD algorithm could act as an early warning system for rare, previously unseen problems.
- calorimeter data of 16-dimensional time series entered into model
- The 16-dimension time series data is obtained from the data pipeline below



- Individual calorimeter signal reading (cell) grouped into topoclusters (standard ATLAS technique to group cell that gives data that represents footprint of a single particle/jet). Two variables are calculated for each cluster (Q factor and timing), topocluster are grouped by which detector partition they belong to (among 4)(EMBA, EMBC, EMECA, EMECC). For each detector partition they find the mean_Qfactor, mean_timing, STD_Qfactor and STD_timing of clusters that lie in it. These $4 \times 4 = 16$ features are normalized and given to the model
- Analysis on data from roughly one-minute intervals of data-taking.
- Only data certified as good for physics analyses by ATLAS was used in the training. Calorimeter detector issues are generally rare, for instance the corresponding data loss during 2015–2018 data-taking was below 1%. The model used is shown below



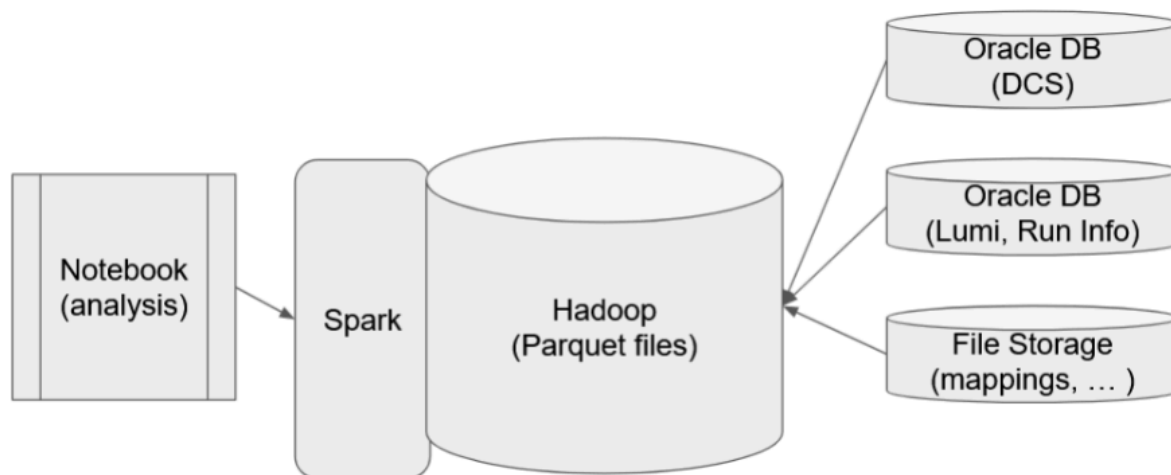
- calorimeter periodically suffers from bursts of coherent noise, last for around millisecond. A well-established algorithm to identify such noisy events already exists within ATLAS and was used to provide ground-truth labels to benchmark the AE performance.



- Y-axis: showing what fraction of events in each category fall at each loss value
- AE reconstruction loss for the events identified as noisy by this algorithm is significantly higher compared to the rest. An anomalous time window is obtained by requiring several such consecutive events in order to reduce the risk of single rare events being flagged as detector issue. AE also found additional anomalous time windows for proton–proton collision data, likely corresponding to a temporary failure of a vacuum diffusion pump that was not found by the existing algorithm.

4) Advancing ATLAS DCS Data Analysis with a Modern Data Platform (research)

- The DCS data, stored in an Oracle database is optimized for transactional operations not for large-scale analysis
- They developed a data pipeline using Apache Spark, CERN's Hadoop service, and the CERN SWAN platform.
- Used it to analyze Data Acquisition (DAQ) links for the ATLAS New Small Wheel (NSW) detector,
- In oracle database, EVENTHISTORY table serves as the central repository, storing sensor IDs, timestamps, and measurement values, with new entries added continuously
- Since EVENTHISTORY table too large, range partitioning is used to divide the table into smaller, manageable segments based on predefined intervals such as monthly partitions
- data import process using Apache Spark jobs scheduled to run daily, which read data from the source Oracle database (ATONR_ADG, the read-only replica of the main production database, ATONR) and write it into the Hadoop platform as Parquet files
- Spark JDBC (Java Database Connectivity) datasource, configured with the Oracle JDBC driver, is used for data transfer
- Data are partitioned by day, with each partition stored as a separate Parquet file
- Currently, 3 TB of DCS data have been imported into the Hadoop ecosystem.(30% of the data)(only records from 2022 onwards)
- CERN's Hadoop cluster, Analytix with 1400 physical cores and 20 PB of distributed storage is used
- Spark's DataFrame (like pandas dataframe library) API, Spark SQL API and Spark Python API (pySpark) offer a high-level interface for data manipulation and analysis



- The front-end analysis is performed using Jupyter notebooks
- analysis framework is built on CERN's SWAN (Service for Web-based Analysis), a Jupyter notebook environment accessible to all CERN user (which integrated with CERNBox for file storage) (preconfigured Spark connector helps in straightforward interaction with CERN's Hadoop service)

- NSW engineers observed DAQ link instability and thought maybe the VTRx (a device that convert light to electric signal which the DAQ link physically depends on) is degrading, and RSSI (the VTRx's diagnostic) should show it(become close to 0.5V). To find it out they looked if for the specific links that had DAQ instability, does the RSSI for those same links have voltage close to 0.5V. They found no correlation.